

Weather Prediction System

Abstract

The aim of the project is to analyze short-term meteorological prediction by training machine learning algorithms on the historical climatological data related to North Carolina. We applied and performed a comparative analysis between a simple linear regression model and a more advanced Lstm model. Experimental results of the first stage show that both models can reproduce the general tendencies of temperature, but the Lstm has slightly higher quality in reproducing the temperature fluctuations. By using these models, we will be able to predict the meteorological parameters such as temperature, humidity, and precipitation. In this comparative analysis, we determine whether a statistically significant improvement in the accuracy of the forecasts is produced by the augmentation of the model complexity.

1 Introduction

Weather forecasting plays a significant role in modern society. Reliable predictions help communities prepare for extreme temperatures, optimize agricultural planning, unexpected events, and manage infrastructure. Traditional forecasting relied on simulation models created by meteorologists, but using machine learning models can improve prediction accuracy.

This project focuses on short-term forecasting (predicting temperature or precipitation) using both a simple linear regression model and a more advanced neural network. We implemented both models and evaluated their performance on real weather data. The goal is to compare their accuracy, understand the trade-offs between simplicity and complexity, and analyze how well each model performs on unseen data.

2 Motivation

Machine learning models deployed in weather forecasting systems influence real-world decisions. Errors in forecasts may lead to incorrect planning or decisions. It can also help people with respiratory illnesses affected by humidity or air quality.

For example Meteorologists issued a rare level 4 out of 5 severe weather warning, in central North Carolina, in March 2026 and said they could expect a tornado outbreak. However, the expected cyclone did not materialize because the thermodynamic conditions required on the surface, i.e. adequate heat and humidity, were not satisfied(1).

It's incidents like this that can impact our lives more

than we believe.

Wrong weather forecasts can lead to flight cancellations, road closures, and with time people will lose trust and display public frustration, which can lead to some individuals ignoring warning even if they are true ... individuals ignoring warning even if they are true.

3 Dataset

We will use a publicly available time-series weather dataset from North Carolina. The dataset contains historical meteorological measurements, including:

- Temperature
- Humidity
- Wind speed
- Atmospheric pressure
- Precipitation

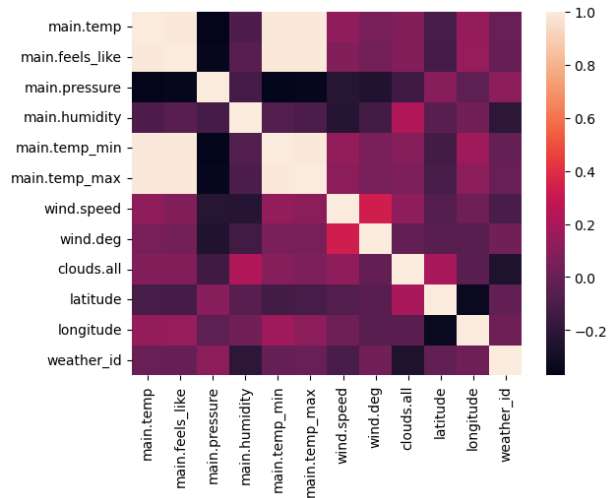


Figure 1: Correlation heatmap of weather features.

The sequentially recorded data are time-series observations and we will preprocess the dataset through normalisation and missing value handling so that we can prepare the data for analysis. The data that was collected used hourly readings from weather stations, satellites, radar.

As depicted in Figure 1 the relationship between the different variables related to weather (main.temp, main.temp min, and main.temp max) have a strong correlation, improving the accuracy of model prediction. Similarly, humidity, wind speed, and cloud even have weak correlations within the dataset but provide valuable alternative information.

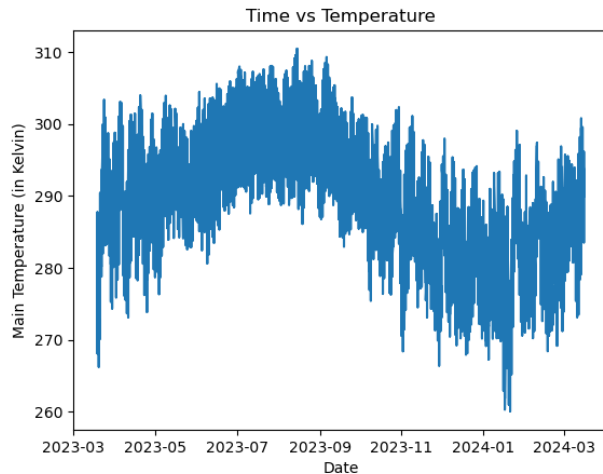


Figure 2: Time vs temperature values.

Figure 2 shows the temperature values over time. The graph shows a clear seasonal pattern where temperatures increase during the middle of the year and decrease toward the end. There are also short-term fluctuations, which make the prediction task more challenging for the models.

The dataset used in this project is publicly available online: HuggingFace Weather Dataset

4 Methodology

4.1 Forecasting Task

The primary task is regression-based forecasting. Given a sequence of past weather measurements, the model will predict future values such as:

- Temperature on a specific day
- Precipitation probability
- Multi-step forecasts (next 6 or 24 hours)

4.2 Data Cleaning

For the data set, it is important to clean parts of the data that isn't needed. Firstly, it is important to drop the "main.feelslike", "main.temppmin" and "main.temppmax" columns since they are derived from the temperature and we are trying to make a model that accurately predicts the temperature.

From there, it is important to look at the significance of categorical variables. For the "Weather.main" variable (shows overall weather such as clear and cloudy), we displayed the count of each unique weather main variable so that we encode the most common ones. We dropped weather such as "Squall" since there is only one entry and can cause noise. Then, We encode it into dummy variables using pandas.

4.3 Feature Engineering

For our time series analysis problem, we researched how to utilize/curate our data for model prediction.

- For time series problems it is important to utilize pandas and convert our date column to a pandas date time data type so that we can make time based variables. We then made columns for day, week, month, quarter, and year so that the model can grasp certain time-based patterns that can help predict temperature. For instance, our day column can show that temperature decreases as it gets into the night hours.
- Since time is cyclical, it is of importance to cyclical encode the data by mapping time values onto a unit circle using sine and cosine transformations.
- Another aspect of time-series forecasting feature engineering that is of importance are lag features. These features take in values from previous time steps (in this case days) so that the model can look at these past values and can help predict future values.
- Finally, since the data takes place in multiple cities of North Carolina, we decided to focus on one city, Asheville, NC. This decision allowed us to subset the data which can make training easier since its for one area.

4.4 Models

• Linear regression

- A simple and interpretable model used as a reference.
- Captures general trends in the data but struggles with sudden changes.

Linear regression serves as a baseline because it is a simpler and clearer to understand model than others. Linear regression models the relationship between input features and the target variable as a straight line. As such, it is quick and easy to train; however, because it models data in such an easily interpreted manner, it lacks the capability of accurately modeling more complex data patterns. This often creates issues in cases where the response variable has an abrupt or rapidly changing value over time or when the response variable is influenced by individual features in a non-linear position.

• Lstm

- A more advanced model designed to capture patterns in time-series data.
- Able to learn local variations and fluctuations better than linear regression.

The Lstm is used to capture more complex patterns in the data. Unlike linear regression, the Lstm can learn local patterns and relationships within sequences of data. This makes it more suitable for time-series forecasting, where nearby data points are often related. The Lstm can better adapt to fluctuations in temperature and capture short-term variations that simpler models may

miss. This model utilizes many small decision trees and since there are many variables that make up the weather

- **XGBoost**

- Third model to test when using a random forest type architecture
- This is also useful for time-series forecasting, like LSTM
- There are many hyper parameter that you can tweak for a given dataset with many features.

XGboost is the last model that we will use for testing because it combines high predictive power with smart optimizations that make it easy to adjust many different parameters. Unlike linear regression ,XGBoost builds a large amount of decision trees and each new tree corrects errors from the previous one, allowing it to learn non-linear relationships, feature interaction, and a more complex decision boundary. We decided XGBoost would be good for our dataset because it is good at handling large datasets. Also it includes lasso and ridge regularization along with tree pruning because it is easy for large models to get overfit.

The main reason that we picked to use XGBoost is for the vast hyper parameter adjusting as it allows us to adjust parameters like: n number of estimators, learning rate, max depth of the tree, and the random state which is important for getting consistent results.

- **Data preprocessing**

- Normalized input features to improve training performance.
- Handled missing values in the dataset.
- Converted data into sequences for time-series prediction.

Performing data preprocessing will enhance the overall performance of your model. Normalizing the data makes it possible to keep the range of all features close together, which allows the model to learn from training better. Managing missing values allows you to eliminate errors, and allows the data set to remain consistent with respect to the rest of the data set. Representing the data in sequences will enable the model to learn from previous examples, and then to project or predict future data.

- **Training and evaluation**

- Models were trained on historical weather data.
- Tested on a separate dataset to evaluate performance.

The overall prediction pipeline consists of preparing the data, training the models, and evaluating their performance. First, the dataset is split into training and testing sets to ensure that the models are evaluated on unseen data.

Then, both models are trained using the same input data to allow for a fair comparison. Finally, predictions are generated and compared with actual values using both visual analysis and error metrics.

4.5 Experimental Results

- **Base model performance**

- Both models were able to follow overall temperature trends.
- Predicted values are generally close to actual values.

To evaluate model performance, we compare predicted values with actual temperature data. Then tested there errors to see how each model will compare to each other, because it might be hard to see the difference just by the graph.

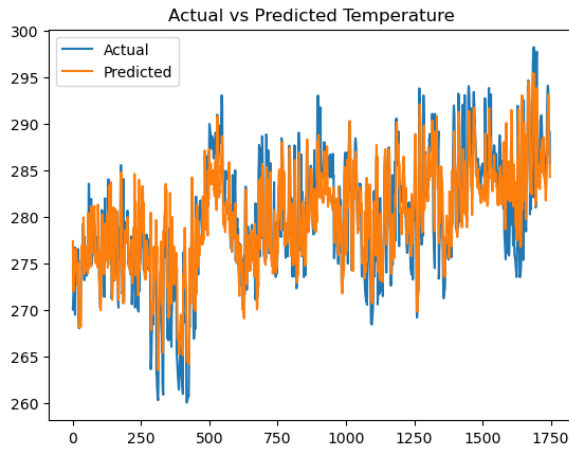


Figure 3: Actual vs predicted temperature values.

Figure 6 shows that the model is able to follow the overall trend of the data, which means it learned useful patterns. However, there are some differences between the predicted and actual values, especially during sudden changes. This indicates that the model is not perfect but still performs reasonably well.

- **Linear regression performance**

- MAE: 2.2511
- RMSE: 3.2161
- R-Squared: 0.7701

Sensitivity to Extremes: The model is sensitive at periods when the system is stable but seems to have an under-shoot with sharp spikes and an over-shoot with sharp temperature drops. Thus, although the general trend is sufficiently described, the reaction of the model can be somewhat weakened, which can also be explained by the normalization processes that are used in the process of preprocessing.

Error Distribution: Residuals (the difference to actual and predicted values) reflect a larger range or difference when quick/sudden changes occur than if there were no quick/sudden changes (as for example in March of 2026 when there was a ‘false alarm’ in N.C.). Sudden deviations (drops/rises) resulting from lack of predictions could lead to dramatic economic/social disruption.

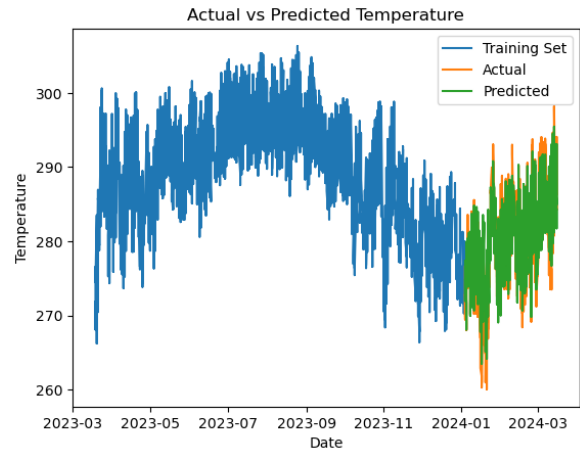


Figure 4: Training data and predicted vs actual temperature values on the test set.

Figure 4 shows the training data along with the model predictions on the test set. The model is able to follow the general pattern of the temperature data, even when applied to unseen data. This indicates that the model has learned useful patterns and can generalize reasonably well. However, some deviations are still present, especially during sudden changes in temperature.

- **LSTM**

- A more advanced recurrent neural network designed specifically to capture long-term dependencies and patterns in time-series data.
- Able to learn local variations and fluctuations better than linear regression through its gating mechanisms (forget, input, and output gates).

The LSTM model was implemented to address the limitations of simpler models in handling sequential weather data. Unlike linear regression, the LSTM can remember information over longer sequences and better adapt to short-term temperature fluctuations. This makes it particularly suitable for time-series forecasting tasks. However, while LSTM showed improvement over the baseline in recognizing local patterns, its performance was ultimately surpassed by XGBoost after hyperparameter tuning.

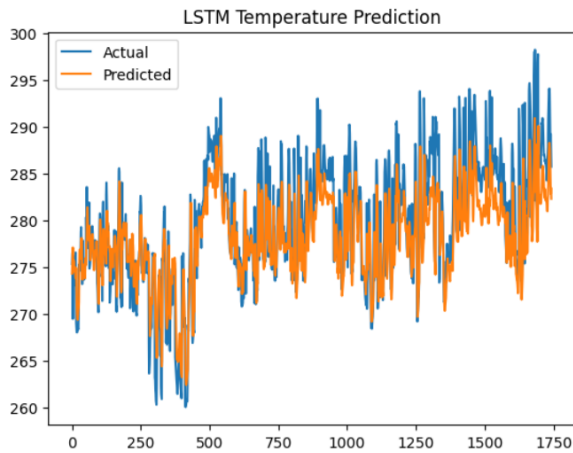


Figure 5: Actual vs predicted temperature values for LSTM

Lstm vs. Baseline: In relation to the baseline linear regression, the Lstm resulted in a moderately higher (but statistically significant) Root Mean Squared Error (RMSE). The ability of Lstm to recognize "local patterns" enables it to be able to better respond to the changes displayed in Figure 3 in a more dynamic manner compared to the linear regression, which has a static structure drawn from a fixed set of parameters. Then when looking at the R-Squared results of this model it actual when up compared to the baseline model so that prompted us to take another approach when picking a model and that lead us to XGBoost.

- **Lstm performance**

- MAE: 2.8211
- RMSE: 3.7623
- R-Squared: 0.6850

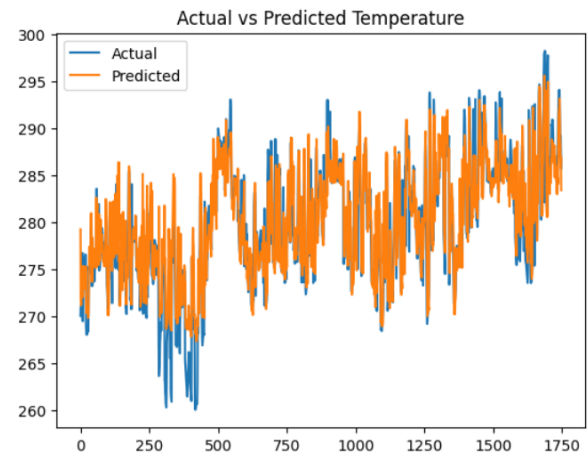


Figure 6: Actual vs predicted temperature values for XGBoost

- **XGBoost performance**

- MAE: 2.0554
- RMSE: 2.9728
- R-Squared: 0.8036

XGBoost compared to the other two models has a improvement in the predictions of the weather. When building this model one of the upsides to it is that there are lots of parameter that could be tweaked. After running many test we came across a result of 200 estimators, a learning rate of 0.05, a max depth of 3, a column sample by tree of 0.8 a reg lambda of 10. This provided a final R-squared value of 0.8036, which when compared to the baseline model is a 0.03 improvement. Also judging base off the high and lows of the day in the figure 5 it is more likely to get close to those limit of the actual data.

Overall, the three models were able to learn the general trends in their respective data sets. However, it appears that the XGBoost has a greater ability to produce predictions that are similar or closer to the actual observed values when there is more variability present in the data. This indicates that XGBoost will be able to better recognize local trends within the time-series data than its counterpart, since when testing on other cities it was not overfit toward the city that the model was trained on.

LSTM was a decent choice, but it did not perform as good compared to the other model with the data possibly needing more complexity in the parameters that this model did not offer. Also the statistical metrics were not as good in this model with the lowest R-squared value of 0.6850.

Linear regression will excel if the data has a very smooth line pattern. Conversely, linear regression won't perform well with sharp spikes or drops in temperature. This limitation exists as linear regression as-

sumes a linear association between the features.

- **Linear regression results**
 - Works well for general trends.
 - Struggles with sudden spikes and drops.
- **Lstm results**
 - Better at capturing fluctuations and local patterns.
 - Produces more accurate predictions overall.
- **XGBoost results**
 - Worked the best and did not overfit to other cities.
 - The multiple parameters allowed for it to be the best overall model.
- **Evaluation metrics**
 - Mean Squared Error (MSE)
 - Visual comparison between predicted and actual values
- **Conclusion from results**
 - Lstm performs better than linear regression.
 - Improvement is moderate, not extremely large.

Overall, the results indicate that model complexity plays an important role in forecasting performance. While simpler models provide a good baseline, more advanced models like XGBoost are better suited for capturing complex patterns in real-world data. However, the improvement is not extremely large, which suggests that there is a trade-off between model complexity and performance. XGBoost seemed to be more fit for this dataset and would be the recommended model if you are trying to predict the weather in North Carolina.

5 Related Work

Recent studies published by EDP Science have applied learning methods such as Support Vector Regression and K-Nearest Neighbors to predict weather temperatures using datasets from England. These studies show that more advanced models can improve prediction accuracy compared to simple regression methods. However, some works do not fully address proper validation techniques for time-series data. In our project, we aim to improve on this by using appropriate evaluation methods and comparing both simple and deep learning approaches.

The referenced study can be accessed at: EDP Science Weather Prediction Study

6 Conclusion

A regression line does not take into account all major variation or fluctuation from linear regression; however, it still has very good predictive value ($R^2 = 0.7670$). For example, if you want to make predictions that are more likely to come out correct, the regression line does not produce

good results, because it does not closely resemble the actual distribution of how things happen in real life.

On the other hand, the neural network model provides more flexibility actual situation that occurs in real life with significantly larger variance between observations. Then when looking at XGBoost is the best of both worlds because it has the complexity with a random forest model, while having a higher R^2 value of 0.8036. This shows that XGBoost is the best model and provided the best outputs.

To summarize, the three models differ in their interpretability, as baseline models are much easier to interpret than complex models. The baseline model also does not show very much improvement over the complex models. The next step would be to investigate the feasibility of creating a hybrid model that uses the simple linear regression model's "clean" style/structure and the flexibility of the XGBoost random forest model to create a hybrid model that could yield more accurate results and decrease the number of significant system errors.

7 Code Availability

The source code for this project, including model implementation, data preprocessing, and experiments, is available on GitHub: CSE 404 Project Repository

8 Citations

1- Skinner, Anna. "Meteorologist Admits to Making 'Spectacularly Wrong' Forecast." *Newsweek*, 17 Mar. 2026, *Newsweek* article on incorrect weather forecast

9 Bios of Researchers

- **Kyle Syron**
 - NetID: syronkyl
 - Department: College of Engineering Computational Data Science
 - Year: Senior
 - Contribution: Data analysis and baseline model development (linear regression and feature evaluation).
- **Abdulmajeed Alanazi**
 - NetID: alanazi6
 - Department: College of Engineering Computer Science
 - Year: Senior
 - Contribution: Conduct evaluation of model performance, including metric analysis (MSE), and comparative analysis between regression and Lstm models.
- **Muath Alanazi**
 - NetID: alanazi9

- Department: College of Engineering Computer Engineering
- Year: Senior
- Contribution: Handle data preprocessing feature selection and preparation for training and testing.

- **Aidan Swenson**

- NetID: swenso4
- Department: College of Engineering Computational Data Science
- Year: Senior
- Contribution: Conduct model evaluation, comparison, and performance analysis.

- **Ashlynn Blanshan**

- NetID: Blansha2
- Department: College of Natural Science Data Science
- Year: Senior
- Contribution: Design and implement the advanced model (Lstm) model.

link to github: https://github.com/Blansha2/CSE_404_Repo/tree/main